

Arquitecturas de Integración para Aplicaciones Distribuidas: Un Análisis Comparativo de Message-Oriented Middleware, Arquitectura Orientada a Eventos, Arquitectura Orientada a Servicios y Microservicios

Danny Ovelar, Emmanuel Rodríguez, Reynaldo Nequi.

Análisis de sistemas

UCOM, Asunción, Paraguay.

dannyovelar@s.ucom.edu.py

albertorodriguez@s.ucom.edu.py

reynaldonequi@s.ucom.edu.py

La integración de sistemas heterogéneos es uno de los retos centrales del arquitecto de TI moderno. Este artículo presenta un análisis comparativo de cuatro estilos de arquitectura utilizados para conectar aplicaciones distribuidas: Message-Oriented Middleware (MOM), Arquitectura Orientada a Eventos (EDA), Arquitectura Orientada a Servicios (SOA/REST) y Microservicios. Se describen sus características principales, las tecnologías y marcos de trabajo representativos (ActiveMQ, Kafka, RabbitMQ, Amazon SQS, Azure Service Bus, Google Cloud Pub/Sub, Spring Integration, Apache Camel, Mule ESB, WSO2, entre otros), así como sus limitaciones y los criterios que un arquitecto debe considerar al elegir entre ellos. El análisis se basa en la revisión de literatura técnica especializada y documentación oficial de proveedores. Los resultados muestran que no existe un estilo universalmente superior: cada uno resuelve un conjunto distinto de restricciones de negocio y de calidad de software, y en la práctica las organizaciones combinan varios de ellos según el contexto.

Palabras clave—arquitectura de software, integración de sistemas, middleware orientado a mensajes, arquitectura orientada a eventos, arquitectura orientada a servicios, microservicios, bus de servicios empresarial.

I. INTRODUCCIÓN

Ninguna aplicación empresarial vive aislada. Un sistema de ventas necesita conversar con el inventario, el sistema de facturación debe enterarse de lo que ocurre en el punto de venta, y una aplicación móvil de atención al cliente necesita datos que residen en un mainframe construido décadas atrás. A esta necesidad de conectar componentes desarrollados de forma independiente, con distintos lenguajes, protocolos y ciclos de vida, se le conoce como integración de aplicaciones empresariales, y constituye uno de los problemas más persistentes en la ingeniería de sistemas distribuidos [1].

Con el crecimiento de la nube, las arquitecturas hiperconectadas y la presión por entregar software con mayor velocidad, las organizaciones han adoptado sucesivas generaciones de estilos arquitectónicos para resolver este problema: primero el middleware orientado a mensajes (MOM) y los buses de servicios empresariales (ESB), después la arquitectura orientada a servicios (SOA),

más adelante la arquitectura orientada a eventos (EDA) y, en los últimos años, los microservicios. Cada estilo surgió como respuesta a las limitaciones prácticas de su predecesor, pero ninguno reemplazó por completo a los demás; hoy conviven en el mismo ecosistema de TI.

El objetivo de este artículo es ofrecer, en un lenguaje accesible para un público técnico no necesariamente especializado en integración, una explicación clara de qué es cada uno de estos estilos, qué problema resuelve, qué limitaciones presenta y qué criterios debería usar un arquitecto de integración para elegir entre ellos. El alcance cubre cuatro grandes bloques: (i) el middleware orientado a mensajes y sus implementaciones más usadas (ActiveMQ, Amazon SQS, Azure Service Bus, Google Cloud Pub/Sub, Kafka y RabbitMQ), junto con los marcos de integración que automatizan patrones de mensajería (Spring Integration, Apache Camel y Camel-K) y los buses de servicios empresariales (Mule ESB, JBoss Fuse, WSO2 y Oracle Service Bus); (ii) la arquitectura orientada a eventos; (iii) la arquitectura orientada a servicios, incluyendo su variante basada en REST; y (iv) los microservicios.

El resto del artículo se organiza siguiendo la estructura IMRyD adaptada a un ensayo de tipo exploratorio: la Sección II describe la metodología de revisión utilizada; la Sección III presenta los resultados del análisis, estilo por estilo, junto con una tabla comparativa; la Sección IV discute los factores de elección y las combinaciones híbridas más comunes en la práctica; y la Sección V resume las conclusiones.

II. METODOLOGÍA

Este trabajo no reporta un experimento original, sino una síntesis de fuentes técnicas publicadas: documentación oficial de proveedores y fundaciones de código abierto (Apache, WSO2, Spring, AWS, Microsoft, Google), literatura de referencia sobre patrones de integración empresarial [1], material especializado sobre arquitectura de software [4], y artículos técnicos comparativos publicados por profesionales del área [7]–[12]. La revisión se organizó alrededor de seis dimensiones de comparación, elegidas por ser las que con mayor frecuencia determinan la decisión de un arquitecto de integración: (a) el tipo y grado de acoplamiento entre los componentes; (b) el modo

de comunicación, síncrono o asíncrono; (c) la escalabilidad y tolerancia a fallos que el estilo permite alcanzar; (d) la complejidad operativa y de gobierno que introduce; (e) el alcance típico de aplicación, es decir si opera a nivel de una aplicación o de toda la empresa; y (f) el nivel de consistencia de datos que puede garantizarse.

Para cada estilo se identificaron además sus tecnologías o productos representativos, con el fin de que el lector pueda relacionar los conceptos abstractos con herramientas concretas que probablemente ya conoce o utiliza. Los hallazgos se organizan primero de forma descriptiva, estilo por estilo, y luego se resumen en una tabla comparativa transversal que sintetiza las seis dimensiones evaluadas.

III. RESULTADOS: ANÁLISIS COMPARATIVO

A. Message-Oriented Middleware (MOM)

El middleware orientado a mensajes es una capa de infraestructura que permite a aplicaciones distribuidas comunicarse enviando y recibiendo mensajes autocontenidos a través de un proveedor central, en lugar de invocarse directamente mediante llamadas síncronas [2]. El proveedor gestiona el almacenamiento, enrutamiento y entrega de los mensajes, lo que desacopla a los emisores de los receptores: ninguno de los dos necesita estar disponible al mismo tiempo para que la comunicación se complete [2]. Este modelo asíncrono aporta tolerancia a fallos, capacidad de amortiguar picos de carga y facilidad para integrar componentes construidos en plataformas o lenguajes distintos [2], [3].

Existen dos patrones básicos de intercambio dentro de un MOM: el modelo punto a punto, basado en colas, en el que cada mensaje es consumido por un único receptor; y el modelo de publicación-suscripción, basado en tópicos, en el que un mensaje puede ser entregado a múltiples suscriptores interesados [1]. Entre las implementaciones más utilizadas se encuentran RabbitMQ, un bróker basado en el protocolo AMQP reconocido por la flexibilidad de su motor de enrutamiento mediante distintos tipos de intercambios; Apache ActiveMQ, orientado al estándar JMS; Apache Kafka, que en sentido estricto es una plataforma de streaming de eventos basada en un log distribuido, ordenado y replicado, lo que le permite a los consumidores reproducir mensajes pasados, una capacidad valiosa para el procesamiento de flujos de datos [11]; y las opciones administradas por proveedores de nube como Amazon SQS, Azure Service Bus y Google Cloud Pub/Sub, que priorizan la simplicidad operativa a cambio de cierto grado de dependencia del proveedor [11].

Sobre esta capa de mensajería operan los marcos de integración, cuyo propósito es implementar de forma reutilizable los patrones descritos en el catálogo clásico de patrones de integración empresarial [1]. Spring Integration extiende el modelo de programación de Spring para modelar canales, transformadores y enrutadores como componentes inyectables, mientras que Apache Camel

ofrece un motor de enrutamiento y mediación con un lenguaje específico de dominio que permite definir rutas de integración en Java, YAML o XML [7]. Camel-K adapta este mismo motor a un modelo de despliegue nativo de Kubernetes, pensado para integraciones ligeras y efímeras en entornos de contenedores. A diferencia de un bus de servicios empresarial, estos marcos se incrustan dentro de la propia aplicación en lugar de operar como una plataforma centralizada independiente, lo que reduce el bloqueo con un proveedor y favorece el despliegue independiente de cada integración [7].

B. Bus de Servicios Empresarial (ESB)

El bus de servicios empresarial es una evolución del MOM que añade, sobre la capa de mensajería, capacidades de enrutamiento inteligente, transformación de formatos, orquestación de servicios y gobierno centralizado [6]. Cuando dos sistemas envían mensajes en formatos distintos, por ejemplo XML y JSON, el ESB traduce el mensaje al formato que el receptor espera, y puede además dividir una solicitud en varias ramas de procesamiento y agregarlas nuevamente antes de responder.

Entre las implementaciones más difundidas se encuentran Mule ESB, de MuleSoft, que soporta JMS, servicios web, JDBC y HTTP, y permite diseñar flujos mediante XML o la herramienta visual AnyPoint Studio; JBoss Fuse, la distribución empresarial de Red Hat construida sobre Apache Camel; WSO2 Enterprise Service Bus, un motor de integración liviano orientado a componentes que además ofrece un amplio catálogo de conectores hacia sistemas comerciales como SAP [5]; y Oracle Service Bus, orientado a entornos corporativos que ya utilizan la pila de middleware de Oracle. El ESB fue durante años la pieza central de las arquitecturas SOA, pero también ha sido objeto de crítica por convertirse en un punto único de fallo, en un cuello de botella de gobierno y en una plataforma costosa de operar cuando crece sin control, lo que impulsó parcialmente la migración hacia estilos más descentralizados como los microservicios.

C. Arquitectura Orientada a Eventos (EDA)

La arquitectura orientada a eventos se apoya en el procesamiento asíncrono mediante procesadores de eventos altamente desacoplados, que disparan eventos y a la vez responden a eventos que ocurren en el resto del sistema [4]. Sus componentes principales son el procesador de eventos (hoy normalmente llamado simplemente servicio), el evento iniciador que llega desde fuera del sistema y desencadena un flujo de trabajo, el evento de procesamiento o evento derivado que un servicio genera para anunciar un cambio de estado, y el canal de eventos, el artefacto de mensajería físico —una cola o un tópico— que transporta esos eventos [4].

Un ejemplo ilustrativo es el de un pedido de un libro en línea: el evento iniciador Place Order llega al servicio de colocación de pedidos, que al completar su trabajo publica el evento derivado Order Placed sin saber de antemano qué

otros servicios, ni cuántos, van a reaccionar a él; en la práctica pueden responder simultáneamente los servicios de pago, de inventario y de notificación [4]. Esta característica —que el emisor desconozca a sus consumidores— es precisamente lo que distingue a un sistema orientado a eventos de uno orientado a mensajes: un evento informa que algo ya ocurrió, mientras que un mensaje es una orden dirigida a un servicio concreto y conocido, como pedirle a un servicio de pagos que aplique un cargo [4]. Esta diferencia también se refleja en la propiedad del canal y del contrato: en los eventos, el emisor es dueño del canal y del contrato, y son los consumidores quienes deben adaptarse a los cambios; en los mensajes ocurre lo contrario, el receptor es quien posee el canal y el contrato [4].

Por su naturaleza asíncrona y desacoplada, la EDA sobresale en tolerancia a fallos, escalabilidad, rendimiento y extensibilidad arquitectónica, ya que un nuevo servicio puede suscribirse a un evento existente sin modificar al emisor original [4]. Sin embargo, es un estilo débil en simplicidad: la ausencia de un orquestador central dificulta el control del orden y la temporización de los eventos, complica el manejo de errores —porque cuando un servicio falla a mitad de un flujo ya iniciado, no siempre está claro quién debe compensar lo ya ejecutado— y hace que toda la solución opere bajo un modelo de consistencia eventual, poco adecuado para procesos que requieren datos exactos en un instante determinado [4]. Por ello, la literatura de referencia recomienda este estilo cuando el negocio reacciona a sucesos (más que cuando responde a solicitudes de un usuario) y cuando los flujos de trabajo son complejos y no deterministas, pero desaconseja usarlo cuando el procesamiento es mayoritariamente síncrono, orientado a solicitudes tipo CRUD, o cuando se necesita control estricto del orden de ejecución [4].

D. Arquitectura Orientada a Servicios (SOA) y REST

La arquitectura orientada a servicios organiza el software como un conjunto de servicios autocontenidos, discretos y reutilizables que en conjunto forman una malla de capacidades de negocio [8]. A diferencia del MOM, que resuelve solo el transporte de mensajes, una arquitectura SOA suele apoyarse en un bus de servicios empresarial que añade enrutamiento, orquestación y gobierno [10]. El ámbito de diseño de SOA es la empresa completa: los servicios se conciben para ser reutilizados por distintas líneas de negocio y sistemas, lo que exige contratos formales, catálogos de servicios y comités de gobierno que aprueban cambios [9], [10]. Los servicios pueden implementarse con protocolos basados en SOAP y WS-*, propios de la generación clásica de SOA, o con estilos más ligeros basados en REST sobre HTTP, que simplifican el consumo de servicios pero conservan el resto de las características de gobierno y reutilización propias de SOA.

El enfoque de SOA resulta apropiado cuando es necesario integrar muchos sistemas heterogéneos —construidos con tecnologías distintas y a menudo

heredados— bajo un modelo de gobierno centralizado, algo habitual en grandes organizaciones con departamentos de TI consolidados [10]. Su principal limitación es que ese mismo gobierno centralizado, y la dependencia de un bus compartido, tiende a introducir cuellos de botella organizacionales y técnicos que reducen la agilidad de los equipos de desarrollo, además de acoplar el ritmo de cambio de un servicio al de todos los demás que comparten el bus [9].

E. Microservicios

Los microservicios llevan la idea de servicios independientes al nivel de la aplicación en lugar de la empresa: cada microservicio encapsula una única capacidad de negocio, se desarrolla, prueba, despliega y opera de manera autónoma, y mantiene su propio almacenamiento de datos [12]. Mientras SOA favorece un enfoque de arriba hacia abajo con gobierno centralizado, los microservicios adoptan un enfoque de abajo hacia arriba, descentralizado, en el que equipos pequeños son dueños de extremo a extremo de su servicio [9]. Otra diferencia relevante es la capa de comunicación: donde SOA típicamente depende de un middleware de mensajería compartido —el ESB—, los microservicios suelen comunicarse a través de una capa de API directa entre servicios, o mediante brokers de eventos ligeros como Kafka o RabbitMQ, sin un componente centralizado que orqueste el tráfico [9], [12].

Esta granularidad fina aporta mayor agilidad, despliegues independientes y aislamiento de fallos —si un microservicio cae, en principio no arrastra al resto del sistema—, lo que resulta especialmente valioso en aplicaciones complejas con requisitos cambiantes [8]. El costo de esa flexibilidad es una mayor complejidad distribuida: gestión de la consistencia de datos entre servicios, pruebas de integración más difíciles, necesidad de observabilidad avanzada y, con frecuencia, equipos con un nivel alto de madurez técnica y de colaboración [8], [12].

F. Comparación transversal

La Tabla I resume las seis dimensiones evaluadas para los cuatro estilos analizados, complementando la descripción cualitativa anterior con una vista comparativa rápida.

Criterio	MOM / ESB	EDA	SOA/REST	Microserv.
Acoplamiento	Bajo	Muy bajo	Medio	Bajo
Comunicación	Asíncrona p2p/pub-sub	Asíncrona por eventos	Sin./asín. SOAP-REST	Sin. vía API, o eventos
Escalabilidad	Alta	Muy alta	Media	Alta
Complej. operativa	Media-alta	Alta	Alta (bus central)	Alta (distribuida)
Alcance típico	Integración de apps	Reacción a sucesos	Empresa (reúso)	App (equipos autónomos)
Consist. de datos	Eventual	Eventual	Fuerte	Eventual

TABLA I. Comparación de los estilos de arquitectura de integración según las dimensiones evaluadas.

IV. DISCUSIÓN: CRITERIOS DE ELECCIÓN

La revisión anterior muestra que la pregunta correcta no es cuál estilo es mejor en abstracto, sino cuál resuelve mejor el conjunto específico de restricciones de una organización en un momento dado. Un primer criterio es la naturaleza del procesamiento: si el negocio reacciona a sucesos —una orden colocada, un pago recibido, un sensor que se activa— la arquitectura orientada a eventos suele ajustarse mejor que un modelo basado en solicitudes; si en cambio el usuario espera una respuesta inmediata a una petición concreta, los estilos síncronos como REST dentro de SOA o microservicios son más naturales [4].

Un segundo criterio es el requisito de consistencia de datos. Los estilos asíncronos —MOM, EDA y, en menor medida, microservicios— operan bajo consistencia eventual, lo cual es aceptable para notificaciones, analítica o flujos tolerantes a la latencia, pero problemático para procesos financieros o regulatorios que exigen datos exactos en el instante de la transacción; en esos casos conviene un estilo orquestado que preserve un mayor control, como SOA con transacciones coordinadas [4], [10].

Un tercer criterio es el alcance organizacional. Cuando el problema es conectar decenas de sistemas heterogéneos —muchos de ellos heredados— bajo políticas de seguridad y auditoría comunes, un ESB dentro de una arquitectura SOA aporta gobierno centralizado que resulta difícil de replicar de otra forma [10]. Cuando el problema es, en cambio, escalar el desarrollo de una única aplicación entre varios equipos pequeños y autónomos, los microservicios evitan el cuello de botella de un bus compartido y permiten desplegar cada componente a su propio ritmo [9], [12].

Un cuarto criterio es la madurez operativa del equipo. La escalabilidad y la extensibilidad que ofrecen la EDA y los microservicios tienen como contrapartida una mayor complejidad de trazabilidad, manejo de errores y observabilidad distribuida; una organización sin la disciplina de monitoreo, pruebas automatizadas y control de versiones de contratos necesaria para sostener esa complejidad probablemente obtendrá mejores resultados iniciales con un ESB tradicional o con servicios SOA más centralizados, aun a costa de menor agilidad [4], [8].

En la práctica, pocas organizaciones adoptan un único estilo de forma pura. Es común encontrar arquitecturas híbridas en las que microservicios internos se comunican de forma síncrona vía REST para operaciones transaccionales, mientras publican eventos de dominio a un backbone basado en Kafka o RabbitMQ para notificar cambios de estado al resto del ecosistema, replicando así los beneficios de extensibilidad de la EDA sin renunciar al control puntual que ofrece una API directa [11], [12]. De igual forma, no es infrecuente que una organización con inversión histórica en Mule ESB o WSO2 mantenga ese bus para la integración con sistemas heredados, mientras adopta microservicios y contenedores para las nuevas

capacidades digitales, usando marcos livianos como Apache Camel o Camel-K como puente entre ambos mundos [7].

V. CONCLUSIONES

Este artículo comparó cuatro estilos de arquitectura para la integración de aplicaciones distribuidas: el middleware orientado a mensajes, el bus de servicios empresarial, la arquitectura orientada a eventos, la arquitectura orientada a servicios y los microservicios. El análisis muestra que estos estilos no son mutuamente excluyentes ni sucesivos en el sentido de que uno reemplace por completo al anterior: cada uno resuelve un conjunto distinto de restricciones de acoplamiento, escalabilidad, consistencia de datos, gobierno y madurez organizacional.

El MOM y el ESB siguen siendo la base más sólida para integrar sistemas heterogéneos bajo un gobierno centralizado; la EDA aporta la mayor extensibilidad y tolerancia a fallos para negocios reactivos, a costa de simplicidad y control; SOA ofrece reutilización a escala empresarial mediante contratos formales; y los microservicios maximizan la agilidad de equipos autónomos dentro del alcance de una aplicación. Para un arquitecto de integración, la recomendación práctica es identificar primero la naturaleza del problema de negocio —reactivo o basado en solicitudes—, el alcance organizacional y la madurez operativa disponible, y a partir de ahí seleccionar, y con frecuencia combinar, los estilos y tecnologías descritos en este artículo.

REFERENCIAS

- [1] G. Hohpe and B. Woolf, "Introduction," Enterprise Integration Patterns. [Online]. Available: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/Introduction.html>
- [2] IEEE Technology Navigator, "Message-oriented middleware." [Online]. Available: <https://technav.ieee.org/topic/message-oriented-middleware/>
- [3] E. Curry, "Message-Oriented Middleware," in *Middleware for Communications*, Wiley, 2004.
- [4] M. Richards, "Chapter 5: Event-Driven Architecture," *Software Architecture Patterns*, 2nd ed., O'Reilly Media.
- [5] WSO2, "What is WSO2 ESB?" [Online]. Available: <https://wso2.com/library/articles/2017/07/what-is-wso2-esb/>
- [6] Wikipedia, "Enterprise service bus." [Online]. Available: https://en.wikipedia.org/wiki/Enterprise_service_bus
- [7] Chakray, "Apache Camel: The definitive guide for integration and EIP." [Online]. Available: <https://chakray.com/apache-camel-the-definitive-guide-to-integrating-applications-and-microservices/>
- [8] BMC Software Blogs, "Service-Oriented Architecture vs Microservices Architecture: Comparing SOA to MSA." [Online]. Available: <https://www.bmc.com/blogs/microservices-vs-soa-whats-difference/>
- [9] Atlassian, "SOA vs Microservices: Which is Best for Your Business." [Online]. Available: <https://www.atlassian.com/microservices/microservices-architecture/soa-vs-microservices>

- [10] O'Reilly, "Chapter 2: Comparing Service Characteristics," Microservices vs. Service-Oriented Architecture. [Online]. Available: <https://www.oreilly.com/library/view/microservices-vs-service-oriented/9781491975657/ch02.html>
- [11] DanubeData, "RabbitMQ vs Kafka vs Amazon SQS: Choosing the Right Message Broker (2026)." [Online]. Available: <https://danubedata.ro/blog/rabbitmq-vs-kafka-vs-sqs-comparison-2026>
- [12] Baeldung on Computer Science, "Microservices vs. Service-Oriented Architecture." [Online]. Available: <https://www.baeldung.com/cs/microservices-soa-differences>